# WorkSpace – Bug/Task Management System

A Spring Boot Based Web Application

**Presented By:** Yash Namdev

B.Tech – Computer Science Engineering
Shri Vaishnav Vidyapeeth Vishwavidyalaya

# Introduction

In modern software development, managing bugs and issues efficiently is essential for delivering high-quality applications. Traditional tracking methods such as spreadsheets or manual communication often leads to confusion and delays.

The **WorkSpace** provides a centralized web platform where teams can create, assign, monitor, and resolve software issues efficiently. The system improves collaboration, transparency, and productivity within development teams.

# Problem Statement

Current challenges in software bug tracking that motivated this project:

### Unorganized Tracking

Manual bug tracking is unorganized and error-prone, leading to lost issues and duplicated work

### No Central System

Lack of centralized issue management system creates fragmented workflows

### Status Visibility

Difficulty in tracking bug status and deadlines results in missed deliverables

### Poor Communication

Team members struggle with poor communication and no real-time visibility of project progress

# Project Objectives

1. **Centralized Platform**

   Develop a unified bug management platform accessible to all team members

2. **Easy Ticket Management**

   Enable users to create and manage tickets with intuitive interface

3. **Real-Time Tracking**

   Provide real-time tracking of bug status and progress updates

4. **Team Collaboration**

   Improve collaboration between developers and teams through structured workflows

5. **Structured Workflow**

   Maintain organized project workflow with clear processes

# System Overview

The Bug Tracker is a web-based application built using Spring Boot architecture. Users can register, authenticate, and manage projects and tickets through an interactive dashboard.

The system follows layered architecture including Controller, Service, and Repository layers to maintain scalability and clean code structure.

# Key Features

### Secure Authentication

User Registration and Secure Login with Email Verification System

### Project Management

Project Creation and Management with organized structure

### Ticket System

Ticket/Bug Creation and Assignment with detailed tracking

### Status Tracking

Real-time tracking with Open, In Progress, Resolved, and Closed states

### Analytics Dashboard

Dashboard for Monitoring Tasks and Progress with user statistics

### User Profile

Profile Page with User Statistics and activity tracking

# Technology Stack

### Backend
Java, Spring Boot, Spring MVC

### Database
MySQL / H2 Database

### ORM Framework
Spring Data JPA & Hibernate

### Frontend
HTML, CSS, JavaScript

### Build Tool
Maven

### Version Control
Git & GitHub

### Containerization
Docker

### Orchestration
Kubernetes

# System Architecture

The system follows a **Monolithic Layered Architecture**:

### Database Layer

Stores user, project, and ticket data with normalized structure

### Entity Layer

Represents database tables with JPA annotations and relationships

### Repository Layer

Handles database operations and data access logic

### Service Layer

Contains business logic and application workflows

### Controller Layer

Handles HTTP requests and REST APIs with request routing

This architecture ensures separation of concerns, maintainability, and scalability.

# Database Design

## Main Entities

01

### User Entity

Stores user credentials, profile details, and authentication information

02

### Project Entity

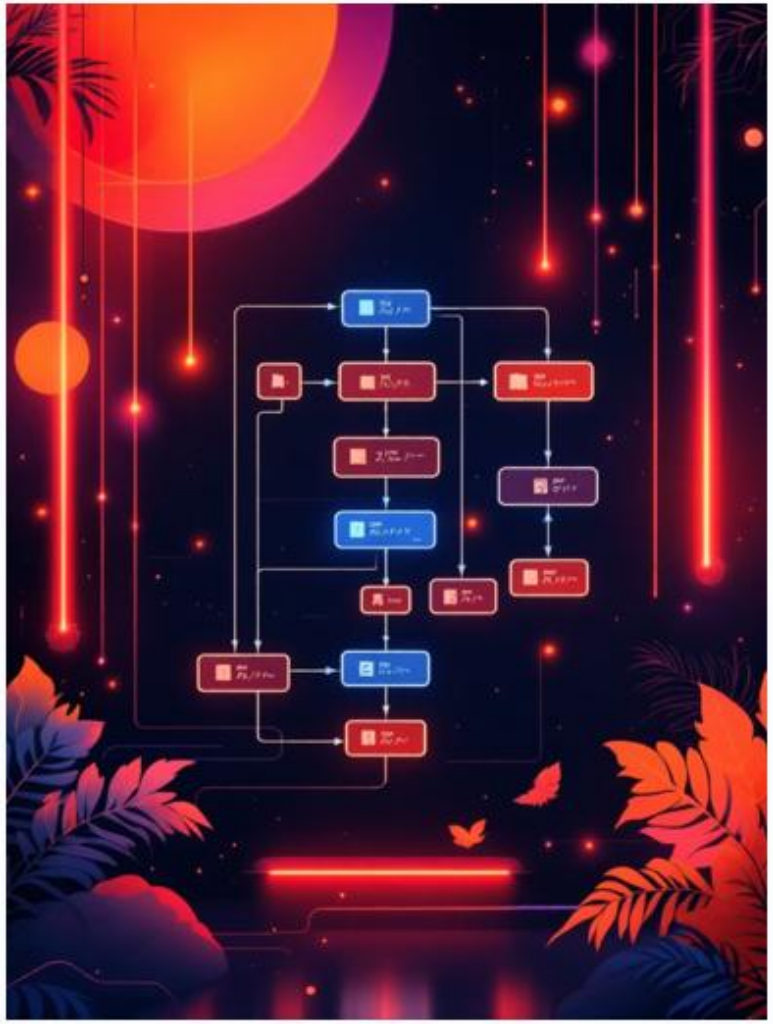
Contains project information, team members, and project metadata

03

### Ticket Entity

Stores bug details such as title, description, priority, status, and assignments

Relationships ensure structured and normalized data storage.



# Docker Deployment

Docker is used to containerize the application to ensure consistency across environments.

### Package

Application packaged into Docker Image with all dependencies

### Containerize

Spring Boot application runs inside container with isolated environment

### Configure

Database container configured separately for data persistence

### Deploy

Easy deployment and scalability across different environments

## Deployment Steps

- Create Dockerfile with application configuration
- Run container using Docker commands
- Build Docker Image using Maven build tools
- Push image to Docker Hub for distribution

# Kubernetes Deployment

Kubernetes builds upon Docker's containerization by providing a powerful framework for orchestrating and managing the entire lifecycle of our containerized bug tracker application.

### Automated Orchestration

Automates the deployment, scaling, and management of application containers, simplifying operations.

### Dynamic Scalability

Automatically scales the application up or down based on traffic and resource demands to maintain performance.

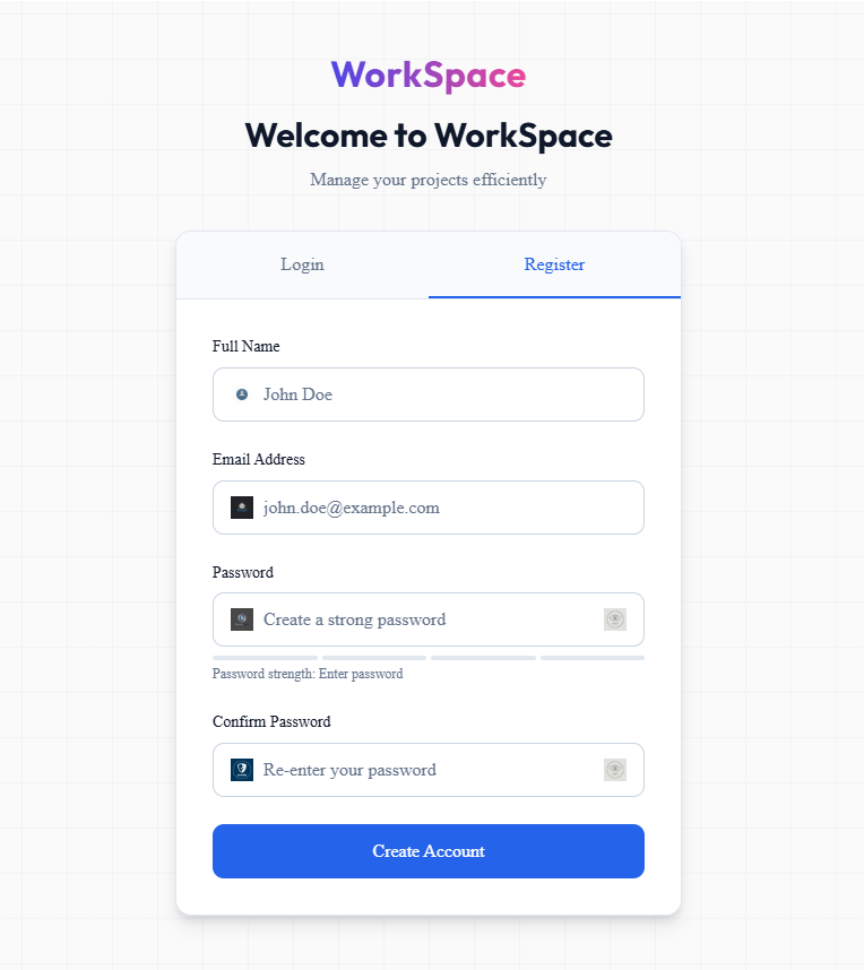
### High Availability

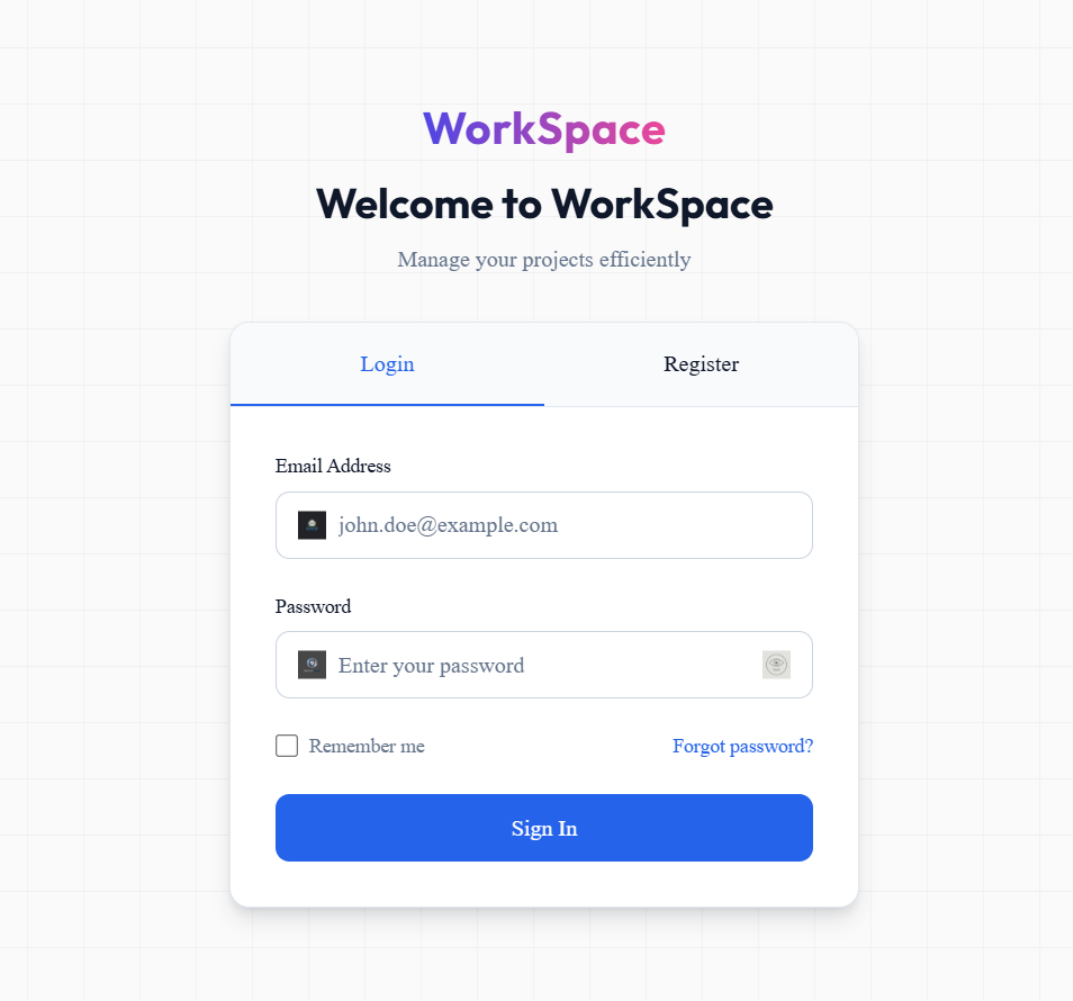
Ensures continuous uptime through self-healing capabilities, automatically restarting failed containers or nodes.

This ensures our Bug Tracker Management System is always available, performant, and resilient.

# Screenshots

Explore the key interfaces of the Bug Tracker Management System:



WorkSpace | Dashboard | My Tasks

| Total Projects | Active Projects | Completed Projects | Average Progress |
|---|---|---|---|
| 1 | 1 | 0 | 0% |

## Your Projects

+ New Project

**trip planner**
it helps in creating trip in collaboraative manner.

Progress 0%

ACTIVE | Open →

---

← WorkSpace | trip planner ACTIVE

Search tickets...

Board
List

TEAM MEMBERS
2 Members
Y Yash Namdev
P Pankaj
+ Add Member

## Project Tickets

+ Create Ticket

| Total Tickets | Open | In Progress | Completed |
|---|---|---|---|
| 4 | 2 | 1 | 1 |

### To Do

TICKET-2
Frontend error
Due:2/27/2026
P Pankaj OPEN

TICKET-3
Database connection
Due:3/2/2026
Y Yash Namdev OPEN

### In Progress

TICKET-1
backend error
Due:2/28/2026
Y Yash Namdev IN PROGRESS

### Done

TICKET-4
home page rendering issue
Due:3/2/2026
P Pankaj RESOLVED

# Future Enhancements

Our roadmap for continuous improvement and expanded capabilities:

### Role-Based Access Control

Implement distinct permissions for Admin, Developer, and User roles to enhance security and operational clarity.

### Real-time Notifications

Integrate WebSockets to provide instant updates on ticket status changes and new assignments.

### File Attachment Support

Enable users to attach relevant files, screenshots, and logs directly to bug reports.

### Advanced Analytics

Develop a comprehensive dashboard offering in-depth reporting and actionable insights into project health and team performance.

### Microservices Migration

Transition to a modular microservices architecture for enhanced scalability, resilience, and independent service deployment.

# Conclusion

The Bug Tracker Management System provides a comprehensive solution for efficient bug tracking and team collaboration, addressing critical pain points in project development.

### Enhanced Efficiency

Streamlined bug reporting and management processes lead to improved project delivery and team productivity.

### Robust & Scalable

Leveraging Spring Boot, Docker, and Kubernetes, the system offers high availability and can effortlessly scale to meet growing demands.

### Future-Ready

Designed with extensibility in mind, it provides a strong foundation for future enhancements and advanced features.

This system is ready to empower your teams with better visibility and control over their projects.

# Thank You

I appreciate your time and attention today. I hope this presentation has provided a clear understanding of our Bug Tracker Management System.

### Feedback & Suggestions

Your insights are invaluable. Please share any feedback or suggestions for further enhancements to the system.

### Questions?

I am ready to address any questions you may have about the architecture, features, or deployment.

### Get in Touch

Feel free to reach out for more detailed discussions or a personalized demonstration of the system's capabilities.